REINFORCE 策略梯度算法在 CartPole-v1 环境上的复现报告

摘要

本文复现了《动手学强化学习》第 9 章中的 REINFORCE 策略梯度算法，并将原始 CartPole-v0 实验改写为适配新版 Gymnasium API 的 CartPole-v1 版本。实验采用一个两层全连接策略网络，根据当前状态输出离散动作概率分布，通过 `Categorical` 分布随机采样动作，并在每个 episode 结束后使用蒙特卡洛回报更新策略网络。实验结果显示，训练初期智能体回报较低，随后随训练逐步上升，后期滑动平均回报稳定在 400–500 附近，说明 REINFORCE 能够成功学习 CartPole 控制任务。与此同时，原始回报曲线波动明显，这也体现了纯蒙特卡洛策略梯度方法方差较大的典型特点。

目录

1 复现目标 **3**

2 算法原理说明 **3**

2.1 策略梯度的基本思想 3

2.2 REINFORCE 的核心公式 4

3 复现环境与运行流程 **4**

3.1 环境安装 4

3.2 运行命令 5

4 代码结构与每个代码块的作用 **5**

4.1 代码块 1：导入库与固定随机种子 5

4.2 代码块 2：滑动平均函数 6

4.3 代码块 3：策略网络 `PolicyNet` 6

4.4 代码块 4：动作采样函数 6

4.5 代码块 5：策略更新函数 7

4.6 代码块 6：训练主循环 8

4.7 代码块 7：绘制结果曲线 9

5 实验结果展示与分析 **10**
5.1 原始回报曲线 10
5.2 滑动平均回报曲线 11

6 结果为什么会抖动 **12**

7 自己的思考与改进方向 **12**
7.1 思考一：REINFORCE 很适合入门理解策略梯度 12
7.2 思考二：只看原始曲线容易误判 12
7.3 思考三：可以加入 baseline 降低方差 12
7.4 思考四：CartPole 成功不代表复杂任务也容易成功 12

8 结论 **13**

A 复现实验中使用的关键超参数 **14**

B 可直接放入报告的结果描述 **14**

1 复现目标

本次复现的目标是把网页中的 REINFORCE 策略梯度算法代码整理成一个可以直接运行的脚本，并在 CartPole-v1 环境中观察训练曲线。核心步骤：

1) 定义策略网络 `PolicyNet`，输入状态，输出动作概率分布；

2) 定义 REINFORCE 智能体，通过动作概率分布随机采样动作；

3) 采集一整条 episode 轨迹；

4) 从 episode 末尾往前计算折扣回报；

5) 用策略梯度损失函数更新策略网络；

6) 记录并绘制每个 episode 的回报曲线。

和原始网页相比，本次复现做了一个调整：原网页使用的是旧版 `gym` 和 `CartPole-v0`，本次复现改为新版 `gymnasium` 和 `CartPole-v1`。新版接口中，`env.reset()` 返回 `state, info`，`env.step(action)` 返回 `next_state, reward, terminated, truncated, info`，因此需要用

```
done = terminated or truncated
```

判断一局游戏是否结束。

2 算法原理说明

2.1 策略梯度的基本思想

强化学习中，智能体在状态 s_t 下选择动作 a_t，环境返回奖励 r_t。策略梯度方法不再先学习一个价值函数再间接得到策略，而是直接学习一个带参数的策略：

$$\pi_\theta(a|s).$$

这里 θ 是神经网络参数，$\pi_\theta(a|s)$ 表示在状态 s 下选择动作 a 的概率。

策略学习的目标是最大化期望回报：

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t r_t\right],$$

其中 γ 是折扣因子，τ 是策略采样得到的一整条轨迹。

2.2 REINFORCE 的核心公式

REINFORCE 使用蒙特卡洛方法估计每个时刻的未来总回报：

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

策略梯度近似为：

$$\nabla_\theta J(\theta) \approx \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t.$$

通俗理解如下：

> 如果某一步选择的动作后面带来了较高回报，那么就提高以后在类似状态下选择该动作的概率；如果某一步选择的动作后面带来了较低回报，那么就降低以后选择该动作的概率。

在 PyTorch 中优化器默认执行的是梯度下降，而我们实际想最大化 $J(\theta)$，所以代码里把损失函数写成负号形式：

$$L(\theta) = -\sum_{t=0}^{T} \log \pi_\theta(a_t|s_t) G_t.$$

这样最小化 $L(\theta)$ 就等价于最大化策略的期望回报。

3 复现环境与运行流程

3.1 环境安装

本次复现建议使用独立 Conda 环境，避免和其他作业环境冲突。

Listing 1：复现环境安装命令

```
conda create -n reinforce python=3.10 -y
conda activate reinforce

pip install torch numpy matplotlib tqdm gymnasium[classic-control]
```

各依赖库的作用见表 1。

表 1：主要依赖库及作用

依赖库	作用
`torch`	搭建策略网络、计算损失函数、执行反向传播和参数更新。
`numpy`	处理数组、计算平均回报、保存训练数据。
`matplotlib`	绘制原始回报曲线和平滑回报曲线。
`tqdm`	显示训练进度条，方便观察当前 episode 和平均回报。
`gymnasium[classic-control]`	提供 CartPole-v1 强化学习环境。

3.2 运行命令

将代码保存为 `reinforce_cartpole.py` 后，在终端中运行：

Listing 2: 训练脚本运行命令

```
python reinforce_cartpole.py
```

脚本运行完成后，会生成两张图片：

- `reinforce_return_raw.png`：每个 episode 的原始回报曲线；
- `reinforce_return_smooth.png`：滑动平均后的回报曲线。

4 代码结构与每个代码块的作用

本次复现代码可以拆成 7 个核心代码块，如图 1 所示。

初始化环境与随机种子 → 定义策略网络 → 定义动作采样 → 采集一整条 episode → 计算折扣回报 → 策略梯度更新 → 绘制训练曲线

图 1: REINFORCE 复现代码整体流程

4.1 代码块 1：导入库与固定随机种子

Listing 3: 导入库与固定随机种子

```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import gymnasium as gym
import matplotlib.pyplot as plt
from tqdm import tqdm


def set_seed(seed=0):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

这一部分的作用是准备训练所需的工具。`random`、`numpy` 和 `torch` 都有自己的随机数生成机制，所以需要同时设置随机种子。这样做的好处是：同样的代码多次运行时，训练趋势更容易复现。

4.2 代码块 2：滑动平均函数

Listing 4: 滑动平均函数

```python
def moving_average(data, window_size=9):
    data = np.array(data)
    result = []

    for i in range(len(data)):
        left = max(0, i - window_size // 2)
        right = min(len(data), i + window_size // 2 + 1)
        result.append(np.mean(data[left:right]))

    return np.array(result)
```

REINFORCE 的原始回报曲线通常非常抖，直接看原始曲线容易误判训练效果。滑动平均的作用是把附近多个 episode 的回报取平均，让整体趋势更清楚。它不改变训练过程，只是用于画图和分析。

4.3 代码块 3：策略网络 PolicyNet

Listing 5: 策略网络 PolicyNet

```python
class PolicyNet(nn.Module):
    def __init__(self, state_dim, hidden_dim, action_dim):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, action_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        action_probs = F.softmax(self.fc2(x), dim=-1)
        return action_probs
```

CartPole-v1 的状态维度为 4，动作数量为 2。因此策略网络完成的是下面这个映射：

$$\text{state} \in \mathbb{R}^4 \longrightarrow \text{action probabilities} \in \mathbb{R}^2.$$

其中 `softmax` 保证两个动作概率非负且总和为 1。也就是说，网络不是直接输出“向左”或“向右”，而是输出：

$$[P(a=0|s), P(a=1|s)].$$

4.4 代码块 4：动作采样函数

Listing 6: 根据策略网络采样动作

```
def take_action(self, state):
    state = torch.tensor(state,
    dtype=torch.float32).unsqueeze(0).to(self.device)
    probs = self.policy_net(state)
    action_dist = torch.distributions.Categorical(probs)
    action = action_dist.sample()
    return action.item()
```

这一段代码体现了策略梯度和普通监督学习的明显区别：智能体不是永远选择概率最大的动作，而是按照概率分布随机采样动作。这样做可以保留探索能力。例如某一时刻网络输出：

$$[P(\text{left}) = 0.7,\ P(\text{right}) = 0.3],$$

它大多数时候会向左，但仍然有一定概率向右，从而继续探索其他可能的策略。

4.5 代码块 5：策略更新函数

Listing 7: REINFORCE 策略更新

```
def update(self, transition_dict):
    states = transition_dict["states"]
    actions = transition_dict["actions"]
    rewards = transition_dict["rewards"]

    G = 0
    loss_list = []

    for i in reversed(range(len(rewards))):
        reward = rewards[i]
        state = torch.tensor(states[i],
    dtype=torch.float32).unsqueeze(0).to(self.device)
        action = torch.tensor(actions[i],
    dtype=torch.long).to(self.device)

        probs = self.policy_net(state)
        action_dist = torch.distributions.Categorical(probs)
        log_prob = action_dist.log_prob(action)

        G = reward + self.gamma * G
        loss = -log_prob * G
        loss_list.append(loss)

    total_loss = torch.stack(loss_list).sum()
    self.optimizer.zero_grad()
    total_loss.backward()
    self.optimizer.step()
```

这是整份代码最核心的部分。它做了三件事：

1) 从 episode 末尾开始往前计算 G_t；

2) 对每一步计算 `-log_prob * G`；

3) 把所有时间步的损失加起来，对策略网络做一次反向传播。

为什么要从后往前算？因为最后一步的未来回报就是最后一步奖励，而倒数第二步的未来回报等于“倒数第二步奖励 + 折扣后的最后一步回报”。所以用反向循环可以高效递推：

$$G_t = r_t + \gamma G_{t+1}.$$

4.6 代码块 6：训练主循环

Listing 8: 训练主循环

```python
for i in range(10):
    with tqdm(total=num_episodes // 10, desc=f"Iteration {i}") as pbar:
        for i_episode in range(num_episodes // 10):
            episode_return = 0
            transition_dict = {"states": [], "actions": [], "rewards": []}

            episode_id = i * (num_episodes // 10) + i_episode
            state, info = env.reset(seed=episode_id)
            done = False

            while not done:
                action = agent.take_action(state)
                next_state, reward, terminated, truncated, info = env.step(action)
                done = terminated or truncated

                transition_dict["states"].append(state)
                transition_dict["actions"].append(action)
                transition_dict["rewards"].append(reward)

                state = next_state
                episode_return += reward

            return_list.append(episode_return)
            agent.update(transition_dict)
            pbar.update(1)
```

这一段是智能体和环境交互的过程。每一局 episode 中，智能体不断观察状态、选择动作、获得奖励并进入下一个状态。等这一局结束后，才调用 `agent.update()` 更新网络。这正是 REINFORCE 的特点：

先完整地玩完一局，再根据这一整局的好坏来更新策略。

4.7 代码块 7：绘制结果曲线

Listing 9: 绘制训练曲线

```
episodes = list(range(len(return_list)))

plt.figure()
plt.plot(episodes, return_list)
plt.xlabel("Episodes")
plt.ylabel("Returns")
plt.title("REINFORCE on CartPole-v1")
plt.savefig("reinforce_return_raw.png", dpi=300)
plt.show()

mv_return = moving_average(return_list, window_size=9)

plt.figure()
plt.plot(episodes, mv_return)
plt.xlabel("Episodes")
plt.ylabel("Returns")
plt.title("REINFORCE on CartPole-v1 - Moving Average")
plt.savefig("reinforce_return_smooth.png", dpi=300)
plt.show()
```

这一部分负责把训练结果可视化。原始曲线可以观察每个 episode 的真实波动情况，平滑曲线可以观察整体训练趋势。报告分析时，更推荐以平滑曲线作为主要依据，同时结合原始曲线说明训练不稳定性。

5 实验结果展示与分析

5.1 原始回报曲线

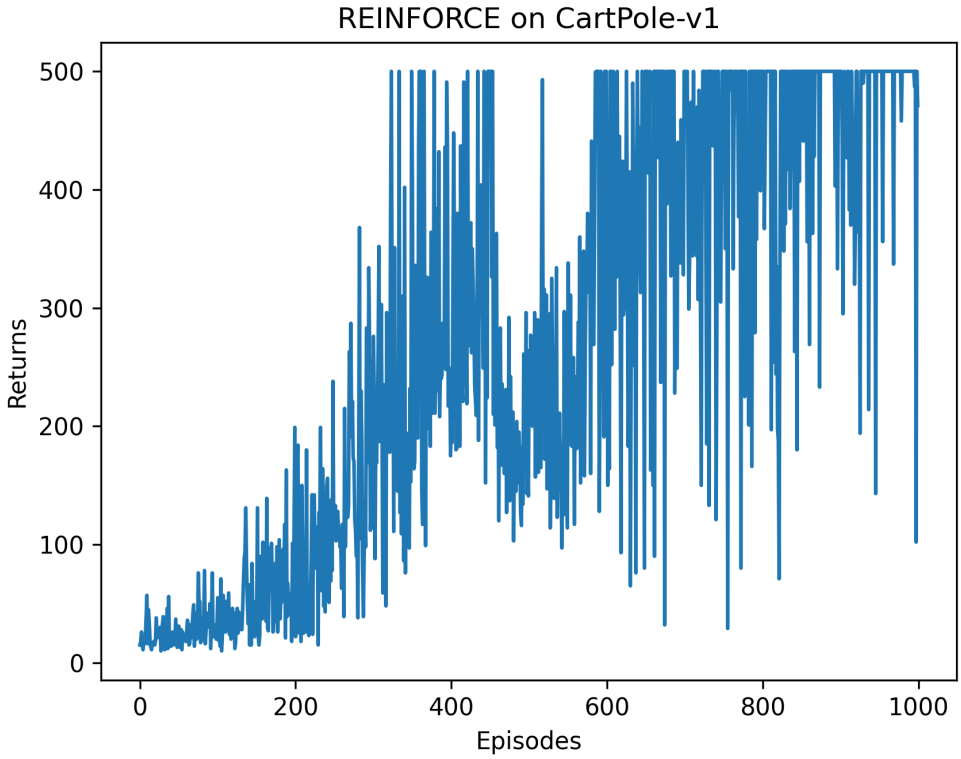


图 2: REINFORCE 在 CartPole-v1 上的原始回报曲线

图 2 展示了每个 episode 的原始回报。可以看出，训练早期智能体基本不会控制小车，回报大多在 20–100 左右；随着训练进行，回报开始快速上升，说明策略网络逐渐学会了在杆子倾斜时采取更合理的推车动作。后期许多 episode 达到了 500 分，说明智能体已经能在不少回合中完成 CartPole-v1 的最长控制时间。

不过，原始曲线中也能看到非常明显的上下波动。例如后期虽然经常达到 500 分，但仍然会偶尔掉到较低回报。这不是代码错误，而是 REINFORCE 算法本身常见的现象。原因主要有两点：

1) REINFORCE 使用一整条轨迹的蒙特卡洛回报，回报估计方差较大；

2) 策略是随机策略，即使已经学得较好，也仍然会按照概率采样动作，因此偶尔会采样到不理想动作。

5.2 滑动平均回报曲线

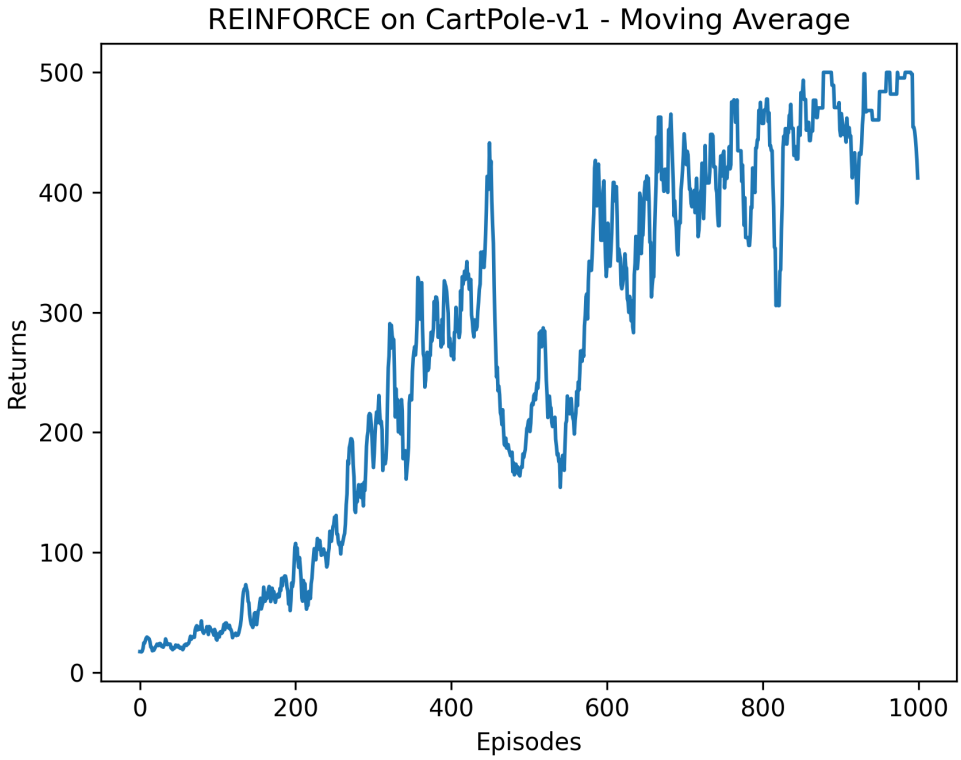


图 3: REINFORCE 在 CartPole-v1 上的滑动平均回报曲线

图 3 更适合用来判断训练是否成功。整体趋势可以分为四个阶段:

表 2: 训练过程阶段分析

训练阶段	现象与解释
0–200 episode	回报较低，智能体主要依靠随机探索，尚未形成稳定策略。
200–450 episode	回报明显上升，说明策略网络开始把高回报动作的概率调大。
450–550 episode	出现明显下降和波动，体现 REINFORCE 蒙特卡洛估计方差较大的问题。
600 episode 以后	回报整体维持在较高水平，多次接近 500，说明策略已经基本学会 CartPole 控制任务。

因此，本次复现实验可以认为是成功的。虽然训练曲线并非严格单调上升，但滑动平均回报最终稳定在高分区间，说明策略梯度方法确实能够通过与环境交互逐步改进策略。

6 结果为什么会抖动

REINFORCE 的优点是思想非常直接，代码也比较简单；缺点是训练方差较大。这里的“方差大”可以用大白话理解为：同样一个策略，因为每一局采样到的轨迹不同，最后得到的回报可能差很多。于是算法有时会根据一条偶然较好的轨迹大幅更新，也可能根据一条偶然较差的轨迹把策略拉偏。

在 CartPole-v1 中，每一步只要杆子还没倒就奖励 1。因此，一条 episode 能坚持多少步，回报就是多少。若一次随机采样的动作序列刚好比较顺利，就会得到很高分；若某几步采样偏了，杆子很快倒下，回报就会突然降低。这就是图 2 中后期仍然有大幅下跌的主要原因。

7 自己的思考与改进方向

7.1 思考一：REINFORCE 很适合入门理解策略梯度

REINFORCE 的最大优点是逻辑清楚：采样一条轨迹，算未来回报，用回报乘以动作概率的对数，再更新网络。它非常适合作为策略梯度的第一版算法来学习，因为它把“好动作概率变大，坏动作概率变小”这件事表达得很直接。

7.2 思考二：只看原始曲线容易误判

原始回报曲线虽然真实，但非常抖。如果只看原始曲线，可能会觉得模型一会儿学会、一会儿又不会。实际上，更合理的判断方式是同时看滑动平均曲线。只要滑动平均曲线总体上升，并且后期维持在高分区间，就说明训练是有效的。

7.3 思考三：可以加入 baseline 降低方差

当前代码使用的是最朴素的 REINFORCE：

$$L(\theta) = -\log \pi_\theta(a_t|s_t)G_t.$$

后续可以加入一个 baseline，把 G_t 改成优势函数形式：

$$A_t = G_t - b(s_t).$$

这样可以减少不同轨迹之间的回报波动，让训练更加稳定。这也是 Actor-Critic 方法继续发展的一个重要动机：用 Critic 估计价值函数，再帮助 Actor 更稳定地更新策略。

7.4 思考四：CartPole 成功不代表复杂任务也容易成功

CartPole-v1 是一个比较简单的控制任务，状态维度只有 4，动作也只有 2 个。因此 REINFORCE 可以在 1000 个 episode 左右学到较好策略。但如果换成状态更复杂、动作连续、奖励更稀疏的任务，单纯 REINFORCE 往往会训练很慢，甚至很难稳定收敛。这也是为什么后续会发展出 Actor-Critic、PPO、SAC 等更稳定的策略优化算法。

8 结论

本次复现完成了 REINFORCE 策略梯度算法在 CartPole-v1 环境上的训练实验。代码层面上，复现脚本包含环境初始化、策略网络定义、动作概率采样、episode 轨迹收集、折扣回报计算、策略梯度更新和结果可视化等模块。实验结果显示，智能体在训练初期回报较低，随后回报逐渐上升，后期滑动平均回报达到 400–500 分附近，说明算法成功学习到了较好的控制策略。

同时，实验也展示了 REINFORCE 的典型不足：由于使用蒙特卡洛回报估计梯度，训练曲线波动明显，策略更新方差较大。因此，REINFORCE 适合理解策略梯度的基本思想，但在更复杂任务中通常需要结合 baseline、价值函数或更稳定的策略优化方法进一步改进。

A 复现实验中使用的关键超参数

表 3: 实验超参数设置

参数	含义	取值
`env_name`	强化学习环境	CartPole-v1
`num_episodes`	总训练 episode 数	1000
`learning_rate`	Adam 优化器学习率	1×10^{-3}
`hidden_dim`	策略网络隐藏层维度	128
`gamma`	折扣因子	0.98
`window_size`	滑动平均窗口大小	9

B 可直接放入报告的结果描述

实验结果表明，REINFORCE 算法能够在 CartPole-v1 环境中逐步学习有效策略。训练初期，智能体主要随机探索，单回合回报较低；随着训练进行，策略网络逐渐提高优质动作的采样概率，回报整体呈上升趋势。在约 600 个 episode 后，滑动平均回报稳定提升至 400 分以上，并多次接近环境上限 500 分，说明策略梯度方法能够成功解决该控制任务。与此同时，原始回报曲线存在较大波动，这是由于 REINFORCE 使用蒙特卡洛回报估计策略梯度，方差较高所导致的典型现象。

参考文献

[1]《动手学强化学习》第 9 章：策略梯度算法，https://hrl.boyuai.com/chapter/2/%E7%AD%96%E7%95%A5%E6%A2%AF%E5%BA%A6%E7%AE%97%E6%B3%95/。

[2] Gymnasium Documentation: CartPole，https://gymnasium.farama.org/environments/classic_control/cart_pole/。